Zephyr Project

IT-Schulungen.com
Technology & Transformation

Praxis 5
ZBus-Stimuli per Shell auslösen

input, pulse und burst als Command-Nachrichten

3.10

Zielbild der Praxis

01	`Shell-Producer`	validiert und publiziert struct event_command
02	`event_cmd_chan`	kopiert und transportiert Befehle mit begrenztem Timeout
03	`Stimulus-Subscriber`	konsumiert, prüft busy und ruft den native_sim-Adapter auf

Die Shell ist Producer. Der Subscriber besitzt Ablaufzustand; der Adapter kapselt Targetdetails.

Arbeitsauftrag in fünf Schritten

Das Source-Code-Projekt liefert Gerüst, Tests und markierte Ergänzungsstellen

01

Vertrag

Enum und Payload für input, pulse und burst

02

Shell publiziert

Argument prüfen und Command senden

03

Subscriber plant

Command konsumieren und Work-Item starten

04

Grenzen prüfen

vor Publikation und im Subscriber absichern

05

zbus-Bilanz

received, rejected und processed vergleichen

Die Praxis endet mit reproduzierbaren Sequenzen und nachvollziehbarer Command- und Event-Bilanz.

event input <0|1>

ZBUS-PRODUCER

```
struct event_command cmd = {
    .type = EVENT_CMD_INPUT,
    .level = level,
};
int rc = zbus_chan_pub(&event_cmd_chan,
                       &cmd, K_MSEC(50));
return shell_publish_result(sh, rc);
```

01 Strikte Eingabe

Nur die vollständigen Werte 0 und 1 werden akzeptiert.

02 Producer-Grenze

Der Handler kennt nur Command-Typ, Payload und zbus-Kanal.

Der Stimulus-Subscriber konsumiert die Nachricht und ruft danach den native_sim-Adapter auf.

event pulse [dauer_ms]

COMMAND-PAYLOAD

```
struct event_command cmd = {
    .type = EVENT_CMD_PULSE,
    .duration_ms = duration_ms,
};
return publish_command(sh, &cmd);
```

01 **Vor Publikation**

Standarddauer und Obergrenze validieren.

02 **Im Subscriber**

Ein delayable Work-Item erzeugt die vollständige aktive Flanke.

Die Shell meldet queued. Ausführung und Abschluss erscheinen anschließend in Logs und Statistik.

event burst <anzahl> <abstand_ms>

COMMAND-PAYLOAD

```
struct event_command cmd = {
    .type = EVENT_CMD_BURST,
    .count = count,
    .gap_ms = gap_ms,
};
return publish_command(sh, &cmd);
```

01 Vor Publikation

Anzahl, Abstand und Überlauf vollständig prüfen.

02 Im Subscriber

Sequenzzustand besitzen und jeden Schritt neu planen.

Die Shell stellt genau eine Burst-Nachricht ein. Der Subscriber erzeugt die Folge schrittweise im Thread-Kontext.

Grenzen und laufende Sequenzen

KONFIGURATION

```
#define MAX_PULSE_MS        ...
#define MAX_BURST_COUNT     ...
#define MIN_BURST_GAP_MS    ...
#define MAX_BURST_GAP_MS    ...
```

01 Parametergrenzen

Benannte Konstanten begrenzen Dauer, Anzahl und Abstand.

02 Konfliktregel

Eine laufende Sequenz wird im Subscriber verworfen; rejected_count steigt.

03 Targetgrenze

Nur der native_sim-Adapter nutzt gpio_emul_input_set_dt(); andere Targets melden unsupported.

Argumentfehler verhindern die Publikation. Fachliche Ablehnung wird über event stats sichtbar.

Prüfmatrix für Pegel, Puls und Burst

Einzelpegel	eine Command-Nachricht; Subscriber setzt den erwarteten Pegel
Puls	eine Command- und eine Input-Nachricht; processed steigt um eins
Burst	eine Command- und N Input-Nachrichten; processed steigt um N
Ungültige Werte	keine Publikation; Command- und Event-Zähler bleiben unverändert
Sequenz aktiv	zweiter Command wird empfangen und im Subscriber als rejected gezählt

event stats zeigt cmd_received, cmd_rejected, input_published und processed.

Abnahme der zbus-Stimuli

FUNKTION

Drei Kommandos

input, pulse und burst publizieren Commands

Subscriber steuert den Adapter

GPIO-Callback liefert Input-Events

ROBUSTHEIT

Definierte Fehlerwege

Argumente vorab geprüft

Publish-Fehler sichtbar

busy als rejected gezählt

Zustand bleibt konsistent

NACHWEIS

Zählerbilanz

Command- und Event-Sollwerte

event stats nach jedem Test

received, rejected und processed

Hardwarestation nicht erforderlich

Bestanden ist die Praxis, wenn Command- und Event-Bilanz für input, pulse, burst und Fehlerfälle den Sollwerten entsprechen.